Subset Sum Problem

Given a set of positive integers and an integer s, is there any non-empty subset whose sum to s.

For example,

```
Input:

set = { 7, 3, 2, 5, 8 }
sum = 14

Output: Yes

subset { 7, 2, 5 } sums to 14
```

Naive algorithm would be to cycle through all subsets of N numbers and, for every one of them, check if the subset sums to the right number. The running time is of order $O(2^N N)$, since there are 2^N subsets and, to check each subset, we need to sum at most N elements.

Custom Searcl

Browse

Adobe Algorithm Amazon BFS Binary Search Bit Hacks Collections DFS FIFO Generics Google Greedy Guava Hashing Intro Java 8 Java 9 JSON Lambda LCS LIFO Maze Memoized Microsoft MIT Must Know Naive Priority Queue Probability Recursive Searching Sliding Window STL

A better exponential time algorithm uses Recursion. Subset sum can also be thought of as a special case of the 0-1 Knapsack problem. For each item, there are two possibilities –

1. We include current item in the subset and recurse for remaining items with remaining sum.

2. We exclude current item from subset and recurse for remaining items.

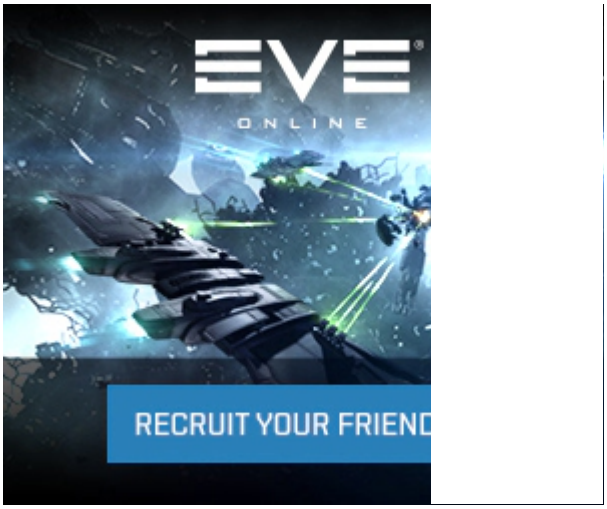
Finally, we return true if we get subset by including or excluding current item else we return false. The base case of the recursion would be when no items are left or sum becomes negative. We return true when sum becomes 0 i.e. subset is found.

Below is C++ and Java implementation of the idea:

C++

```
#include <iostream>
using namespace std;

// Return true if there exists a
bool subsetSum(int arr[], int n,
{
    // return true if sum becomes
    if (sum == 0)
        return true;

    // base case: no items left o
    if (n < 0 || sum < 0)
        return false;

    // Case 1. include current it
    // for remaining items (n - 1
    bool include = subsetSum(arr,

    // Case 2. exclude current it
    // remaining items (n - 1)
    bool exclude = subsetSum(arr,
```

Streams Tabulation Tricky Trie



Subscribe to new posts

Subscribe to receive notifications of new posts by email.

Email Address

Subscribe

```

    // return true if we can get
    // current item
    return include || exclude;
}

// Subset Sum Problem
int main()
{
    // Input: set of items and a
    int arr[] = { 7, 3, 2, 5, 8 }
    int sum = 14;

    // number of items
    int n = sizeof(arr) / sizeof(

    if (subsetSum(arr, n - 1, sum
        cout << "Yes";
    else
        cout << "No";

    return 0;
}
```

Download

Run Code

Output:

```
Yes
```

Java

```
class SubsetSum
{
    // Return true if there exist
    public static boolean subsetS
    {
        // return true if sum bec
        if (sum == 0) {
            return true;
        }

        // base case: no items le
        if (n < 0 || sum < 0) {
            return false;
        }

        // Case 1. include curren
        // for remaining items (n
        boolean include = subsetS

        // Case 2. exclude curren
```

```
        // remaining items (n - 1
        boolean exclude = subsetS

        // return true if we can
        // current item
        return include || exclude
    }

    // Subset Sum Problem
    public static void main(Strin
    {
        // Input: set of items an
        int[] A = { 7, 3, 2, 5, 8
        int sum = 14;

        if (subsetSum(A, A.length
            System.out.print("Yes
        }
        else {
            System.out.print("No"
        }
    }
}
```

Download

Run Code

Output:

```
Yes
```

The time complexity of above solution is exponential and auxiliary space used by the program is `O(1)`.

The problem has an optimal substructure. That means the problem can be broken down into smaller, simple “subproblems”, which can further be divided into yet simpler, smaller subproblems until the solution becomes trivial. Above solution also exhibits overlapping subproblems. If we draw the recursion tree of the solution, we can see that the same sub-problems are getting computed again and again.

We know that problems having optimal substructure and overlapping subproblems can be solved by using Dynamic Programming, in which subproblem solutions are *Memo*ized rather than computed again and again. Below *Memo*ized version follows the top-down approach, since we first break the problem into subproblems and then calculate and store values.

Memoized C++/Java implementation –

C++

```
#include <iostream>
#include <unordered_map>
using namespace std;

// create a map to store solution
unordered_map<string, bool> looku

// Return true if there exists a
bool subsetSum(int arr[], int n,
{
    // return true if sum becomes
    if (sum == 0)
        return true;

    // base case: no items left o
    if (n < 0 || sum < 0)
        return false;

    // construct a unique map key
    string key = to_string(n) + "

    // if sub-problem is seen for
    // store its result in a map
    if (lookup.find(key) == looku
    {
        // Case 1. include curren
        // for remaining items (n
        bool include = subsetSum(

        // Case 2. exclude curren
        // remaining items (n - 1
        bool exclude = subsetSum(

        // assign true if we can
        // current item
```

```
        lookup[key] = include ||
    }

    // return solution to current
    return lookup[key];
}

// Subset Sum Problem
int main()
{
    // Input: set of items and a
    int arr[] = { 7, 3, 2, 5, 8 }
    int sum = 14;

    // number of items
    int n = sizeof(arr) / sizeof(

    if (subsetSum(arr, n - 1, sum
        cout << "Yes";
    else
        cout << "No";

    return 0;
}
```

Download

Run Code

Output:

```
Yes
```

Java

```
import java.util.HashMap;
import java.util.Map;

class SubsetSum
{
    // Return true if there exist
    public static boolean subsetS

    {
        // return true if sum bec
        if (sum == 0) {
            return true;
        }

        // base case: no items le
        if (n < 0 || sum < 0) {
            return false;
        }

```

```
        // construct a unique map
        String key = n + "|" + su

        // if sub-problem is seen
        // store its result in a
        if (!lookup.containsKey(k
        {
            // Case 1. include cu
            // for remaining item
            boolean include = sub

            // Case 2. exclude cu
            // remaining items (n
            boolean exclude = sub

            // assign true if we
            // current item
            lookup.put(key, inclu
        }

        // return solution to cur
        return lookup.get(key);
    }

    public static void main(Strin
    {
        // Input: set of items an
        int[] A = { 7, 3, 2, 5, 8
        int sum = 14;

        // create a map to store
        Map<String, Boolean> look

        if (subsetSum(A, A.length
            System.out.println("Y
        } else {
            System.out.println("N
        }
    }
}
```

Download

Run Code

Output:

```
Yes
```

The time complexity of above solution is `O(n x sum)`. Auxiliary space used by the program

is also `O(n x sum)`.

We can also solve this problem in bottom-up manner. In the bottom-up approach, we solve smaller sub-problems first, then solve larger sub-problems from them. The following bottom-up approach computes `T[i][j]`, for each `1 <= i <= n` and `1 <= j <= sum`, which is true if subset with sum j can be found using items up to first i items. It uses value of smaller values i and j already computed. It has the same asymptotic run-time as Memoization but no recursion overhead.

Below is C++ and Java implementation of the idea:

C++

```
#include <iostream>
using namespace std;

// Return true if there exists a
bool subsetSum(int arr[], int n,
{
    // T[i][j] stores true if sub
    // using items up to first i
    bool T[n + 1][sum + 1];

    // if 0 items in the list and
    for (int j = 1; j <= sum; j++
        T[0][j] = false;

    // if sum is zero
    for (int i = 0; i <= n; i++)
        T[i][0] = true;

    // do for ith item
    for (int i = 1; i <= n; i++)
    {
        // consider all sum from
        for (int j = 1; j <= sum;
        {
            // don't include ith
            if (arr[i - 1] > j)
```

```
                T[i][j] = T[i - 1
            else
            // find subset with s
                T[i][j] = T[i - 1
        }
    }

    // return maximum value
    return T[n][sum];
}

// Subset Sum Problem
int main()
{
    // Input: set of items and a
    int arr[] = { 7, 3, 2, 5, 8 }
    int sum = 18;

    // number of items
    int n = sizeof(arr) / sizeof(

    if (subsetSum(arr, n, sum))
        cout << "Yes";
    else
        cout << "No";

    return 0;
}
```

Download

Run Code

Output:

```
Yes
```

Java

```
class SubsetSum
{
    // Return true if there exist
    public static boolean subsetS
    {
        int n = A.length;

        // T[i][j] stores true if
        // with using items up to
        boolean[][] T = new boole

        // if sum is zero
        for (int i = 0; i <= n; i
            T[i][0] = true;
        }
```

```

        // do for ith item
        for (int i = 1; i <= n; i
        {
            // consider all sum f
            for (int j = 1; j <=
            {
                // don't include
                if (A[i - 1] > j)
                    T[i][j] = T[i
                }
                else {
                    // find subse
                    // the ith it
                    T[i][j] = T[i
                }
            }
        }

        // return maximum value
        return T[n][sum];
    }

    public static void main(Strin
    {
        // Input: set of items an
        int[] A = { 7, 3, 2, 5, 8
        int sum = 18;

        if (subsetSum(A, sum))
            System.out.println("Y
        else
            System.out.println("N
    }
}
```

Download

Run Code

Output:

```
Yes
```

Thanks for reading.

Please use ideone or C++ Shell or any other online compiler link to post code in comments.

Like us? Please spread the word and help us

grow. Happy coding 🙂

Get great deals at Amazon

Sharing is caring:

Tweet Share 26 Share 15 G+

Array, Dynamic Programming

Adobe, Amazon, Memoized, Naive, Recursive, STL, Tabulation

← Partition problem | Dynamic Programming Solution

Minimum Sum Partition Problem →

Leave a Reply

b i link b-quote u ul ol li code spoiler

Join the discussion

Subscribe newest oldest most voted

Surya

Guest

Wonderful

+ 1 − Reply

3 months ago

Surbhi Jain

Guest

Hi,
I am not able to figure out where is the property of Overlapping subproblem in this problem. Would this property always be true here or only in same cases. Where are the subproblems getting overlapped?
Thanks

+ 0 − Reply

3 months ago

Techie Delight

Author

Hi Surbhi, Thanks for sharing your concerns. The overlapping subproblem property is difficult to visible for this problem due to small input size. Consider below program where we have modified the code to print the calls for larger input. https://ideone.com/UBJmSv

Around 40% of the calls are repeated, You can verify that by removing duplicates from the output using below link – Tool: Remove Duplicate Lines

Hope this helps. Please let us know if you have any more concerns regarding this.

+ 1 − Reply

3 months ago

Guest

Surbhi Jain

Thanks for such an elaborate reply. I got your point. One more thing, in the memoized version, you used a map instead of a 2D array. Is it because it is memized and it is possible that if we use a 2D array, all slots would not get filled, so using maps save space?
Thanks a lot

+ 0 −

Reply

3 months ago

Author

Techie Delight

Yes, you're right. In C++/Java, the preferred way is to use map to save space.

+ 0 −

Reply

3 months ago

Guest

Surbhi Jain

Ok. Thanks a lot! 🙂

+ 0 −

Reply

3 months ago

Guest

anshul

HI – very nice explanation. One question – in the bottom up approach, why do we do this:

```
// if sum is zero
for (int i = 0; i <= n; i++)
T[i][0] = true;
```

If you could explain the reasoning – my understanding is since sum

is 0, means there cannot be any subset that can sum up to 0 – so should be false.

+ 0 − Reply

1 month ago

Guest

anshul

I think I got it – the table is storing the results bottom up – so when the sum is zero – means we found a subset where item values equal to 0, so we return true. Correct?

+ 0 − Reply

1 month ago

Guest

ansh ul

*

correction … sum – (item values) equal to 0

+ 0 −

Reply

1 month ago

Techie Delight © 2018 All Rights Reserved. Privacy Policy Contact us